Этап 1. Постановка задачи

№	Сущность / поток	Хранилище	Характер нагрузки	Обоснование	Операции	Спорные моменты
1	Пользователи	PostgreSQL	Средняя частота записи пользователей, поиск по ID	Нужны связи с ролями и целостность, легко поддерживать уникальность и транзакции при регистрации	Регистрация,вход, редактирование профиля	Можно положить профили в MongoDB, но большой шанс потерять целостности и сложности с ролями
2	Сессии и токены	Redis	Частый доступ, высокая нагрузка на чтение и запись	Есть встроенный TTL, атомарные операции	Создание, проверка, истечение по TTL	Можно использовать JWT без северного хранения, но тогда сложнее мгновенно отзывать сессии
3	Курсы и уроки	MongoDB	Средняя частота записи авторов, частое чтение студентов + вложенная структура	Гибкая схема (для разных курсов разные поля)	Создание авторов, редактирование, публикация	Можно нормализовать в PostgreSQL, но тогда будет сложнее работать с производными полями
4	Прогресс по курсам	PostgreSQL + Redis (хеш процесса)	Много записей, частое чтение	В PostgreSQL хранение истории и результатов с индексами, а Redis для быстрого ориентирования	Отметка пройденных уроков, результаты тестов	Только Redis потеряет историю, а только PostgreSQL будет медленный при большом количестве пользователей
5	Поиск по курсам	PostgreSQL	Частое чтение, редкая запись (при изменении курса)	PostgreSQL имеет встроенный полнотекстовый поиск и тригаммы для опечаток, для среднего объема этого будет достаточно	Поиск по названию и описанию, фильтры, поиск с опечатками, ранжирование	Если будет много курсов и понадобятся сложные ранжирования, то тогда лучше Elasticsearh
6	Форум / обсуждения	PostgreSQL/ MongoDB	Древовидная структура	PostgreSQL с рекурсивными данными для деревьев + удобно считать лайки MongoDB для вложенных документов	Создание тем, ответы, лайки	Оба варианта хорошо работают для сложных запросов, но MongoDB можно использовать если понадобиться хранить всю ветку в одном доке

7	Логи действий	ClickHouse	Огромный объем записи, редкие чтения (для аналитики)	Высокая скорость вставки, сжатие, быстрые агрегаты по времени	Поток событий, клики просмотры уроков, начало/ завершение тестов	Можно писать в Kafka, а потом в ClickHouse PostgreSQL и MongoDB не выдержит такой нагрузки
8	Аналитика и отчёты	ClickHouse	Тяжелые агрегаты по историческим данным, редкие но ресурсоемкий запросы	Запросы на агрегаты выполняются быстро	Вся аналитика и отчеты	Можно хранить сырые данные в ClickHouse, а аналитику кэшировать
9	Рекомендации курсов	PostgreSQL + Redis	Связи между пользователям и и курсами, чтение при формировании рекомендаций	В PostgreSQL удобно хранить таблицы В Redis готовый список рекомендаций для пользователя	Похожие курсы, популярное и рекомендации	Для сложных коллаборативный фильтров лучше граммовая БД или отдельный сервис для рекоминдаций

Этап 2. Анализ сущностей

№	Сущность	Часто читают или пишут?	Нужны транзакции?	Нужны JOIN / связи?	Объём данных?	Требования к задержке?	Гибкость схемы?
1	Пользователи	Читают	Да: регистрация + назначение роли одной операцией	Да: роли, права, авторство курсов	Малый: 10^5–10^6 строк, десятки МБ	Проверка прав < 10 мс	Низкая, схема стабильна
2	Сессии и токены	Читают очень часто, пишут при входе	Нет	Нет, только ссылка на user_id	Малый, равен числу активных сессий	< 1–5 мс	Низкая, 3–5 полей
3	Курсы и уроки	Читают часто (каталог), пишут редко (авторы)	Слабые: публикация должна быть атомарной	Да для метаданных (автор, категория), нет для тела курса	Малый по строкам, «тяжёлый» по документам	50–200 мс, кэшируемо	Высокая: разные типы курсов — разный набор полей, вложенность 4 уровня

4	Прогресс по курсам	Пишут часто, читают ещё чаще	Да: результат теста → зачёт урока → сертификат	Да: users, lessons, courses	Большой: users × lessons, 10^7–10^8 строк	Чтение текущего состояния < 50 мс	Низкая, схема жёсткая
5	Поиск по курсам	Только читают	Нет	Нет, документ денормализован	Малый (равен каталогу), но индекс объёмный	< 200 мс	Средняя: маппинг меняется редко, но требует переиндексации
6	Форум / обсуждения	Читают	Слабые: ответ + счётчик, действия модератора	Да: дерево ответов, авторы, темы	Средний: 10^6–10^7 сообщений	200–500 мс допустимо	Низкая
7	Логи действий	Пишут потоком, читают редко	Нет	Нет, события денормализованы	Огромный: 10^9+ строк в год	Запись асинхронная, потеря единичных событий терпима	Средняя: набор полей события расширяется
8	Аналитика и отчёты	Пишут, читают для проверки	Нет	Агрегаты + JOIN с небольшими справочниками	Вся история	Секунды–минуты, свежесть T+15 мин	Низкая, витрины фиксированы
9	Рекомендации курсов	Читают часто (выдача), пишут при пересчёте	Нет	Связи «пользователь → курс» важны при расчёте	Средний	Выдача < 50 мс	Низкая: ключ → список id

Этап 3. Выбор хранилищ и заполнение Data Placement Map

№	Сущность / поток	Операции	Характер нагрузки	Хранилище	Обоснование	Источник истины	Кэш	Спорные моменты
1	Пользователи	Регистрация, вход, редактирование профиля, проверка прав	Read-heavy, малый объём	PostgreSQL + Redis	ACID: пользователь и роль создаются одной транзакцией. Есть FK и JOIN к ролям. Redis снимает горячее чтение профиля на каждом запросе.	Да, PostgreSQL	profile 15 мин, perm 5 мин	Кэш прав → лаг при отзыве доступа
2	Сессии и токены	Создание, проверка, истечение по TTL	Очень частое чтение, короткая жизнь	Redis	Нативный TTL закрывает истечение без крона. Доступ по ключу < 1 мс. Потеря при перезапуске = повторный вход.	Да, Redis	Само хранилище и есть кэш	Refresh-токены: держать ли в PostgreSQL
3	Курсы и уроки	Создание, редактирование, публикация	Write-rare / read-often	PostgreSQL + MongoDB + Redis	Метаданные связаны FK — им нужна целостность. Тело курса — дерево с разным набором полей у разных типов курсов, в реляционной схеме это EAV.	Да, PG — метаданные, Mongo — контент	course: pub 10 мин	Разделение источника истины между двумя базами
4	Прогресс по курсам	Отметка уроков, результаты тестов	Write-heavy + частое чтение	PostgreSQL + Redis + ClickHouse	Результат теста влияет на зачёт и сертификат — нужна транзакционность. Объём большой, но структура жёсткая — спасает партиционирование.	Да, PostgreSQL	progress 1 час, write-through	Рост без лимита; порог перехода на партиционирование

5	Поиск по курсам	Полнотекстовый поиск, фильтры	Read-only, переиндексация по событию	Elasticsearch	Нужны опечатки, стемминг, ранжирование — работа инвертированного индекса. Источник истины остаётся в PG/Mongo, индекс полностью пересобираем.	Нет, производный	Отдельный кэш не нужен	Elasticsearch vs PostgreSQL FTS
6	Форум / обсуждения	Темы, ответы, лайки	Read > write, moderate volume	PostgreSQL + Redis	Дерево ответов ложится на ltree и рекурсивные CTE. Модерация требует целостности и аудита. Лайки — частая мелкая запись без требования точности.	Да, PostgreSQL	likes, сброс раз в минуту	Точность счётчиков при падении Redis
7	Логи действий	Клики, просмотры, старт/финиш тестов	Write-heavy поток, редкие чтения	ClickHouse + буфер	События неизменяемы, пишутся потоком — профиль OLAP. Колоночное сжатие и партиционирование по дням дают DROP PARTITION вместо DELETE.	Да, ClickHouse	Не кэшируется	Kafka или Redis Streams на старте
8	Аналитика и отчёты	Завершения за месяц, воронка по урокам	Редкие тяжёлые агрегаты по истории	ClickHouse (витрины)	Тяжёлые агрегаты нельзя пускать в OLTP-базу — они конкурируют за ресурсы. Справочники реплицируются из PostgreSQL как словари.	Нет, производные	Готовые отчёты 5 мин	Свежесть витрин vs стоимость пересчёта
9	Рекомендации курсов	«Похожие курсы», «популярное у похожих»	Батчевый расчёт + частая выдача	ClickHouse + Redis	Матрица «пользователь × курс» уже лежит в ClickHouse. Результат — короткий список id, выдаём из Redis Sorted Set за < 5 мс.	Нет, производные	rec:user 24 часа	Neo4j vs расчёт в ClickHouse

Вопросы:

1. Где данные создаются впервые? (источник истины)

PostgreSQL — пользователи, метаданные курсов, прогресс, форум. MongoDB — тело курса (модули, уроки, материалы). Redis — сессии. ClickHouse — события. Всё остальное (поисковый индекс, витрины, рекомендации, кэши) производно и восстанавливается пересчётом.

2. Какие данные можно кэшировать? С каким TTL?

Ключ	Что	TTL	Инвалидация
profile:{user_id}	Профиль пользователя	15 мин	При UPDATE профиля
perm:{user_id}	Роли и права	5 мин	При смене роли — принудительный DEL
course:pub:{course_id}	Опубликованная версия курса (метаданные + структура)	10 мин	По событию course.updated
progress:{user_id}:{course_id}	Хеш пройденных уроков и % завершения	1 час	Write-through при отметке урока
likes:{post_id}	Счётчик лайков	без TTL, сброс в PG раз в минуту	—
rec:user:{user_id}	Sorted Set рекомендаций	24 часа	Перезапись при ночном пересчёте

Права кэшируем на 5 минут осознанно: это компромисс между нагрузкой и лагом отзыва доступа, для критичных действий (удаление курса, выплаты) права перечитываются из PostgreSQL напрямую.

3. Какие данные нужны для аналитики? Это те же данные или производные?

Производные. Аналитика работает с фактами (события из ClickHouse, снимки прогресса) и справочниками (пользователи, курсы), скопированными из PostgreSQL. Оперативные таблицы напрямую не читаются никогда.

4. Где нужны ACID-гарантии, а где достаточно eventual consistency?

ACID: регистрация и роли, публикация курса, зачёт теста и выдача сертификата, действия модератора. Eventual consistency: поисковый индекс (лаг секунды), витрины (T+15 мин), рекомендации (T+1 день), счётчики лайков, кэши.